53. Infrastructure as Code (IaC): Terraform ve Pulumi'ye AI Bakışı

Bulut Konsolunda Elle Tıklamayı Bırakıp Altyapıyı Kod Olarak Sürümlemek
Bölüm 9 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"AWS konsolunda fareyle tıklayarak sunucu açmak (ClickOps) neden şirketler için saatli bir bombadır ve Terraform altyapıyı nasıl silinip yeniden kurulabilir kılar?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 ClickOps (Konsolda elle tıklayarak altyapı kurmak) belgelenemez, tekrarlanamaz ve insan hatasına davetiye çıkarır. - 📌 Infrastructure as Code (IaC), sunucuları, veritabanlarını ve güvenlik duvarlarını tıpkı uygulama kodu gibi Git'te versiyonlamaktır. - 📌 Terraform'un `plan` komutu, gerçek buluta dokunmadan önce hangi kaynakların oluşturulacağını veya silineceğini önizleme imkanı sunar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir şehir kurduğunuzu hayal edin.

Belediye başkanı sokakları, su borularını ve elektrik hatlarını mühendislerin aklına estikçe elle kazarak yapmasına izin verirse ne olur? 6 ay sonra kimse hangi borunun nereden geçtiğini bilmez! Yeni bir bina yapmak için kazmayı vurduğunuz an ana su borusunu patlatırsınız! İşte konsolda elle tıklayarak sunucu kurmak (**ClickOps**) tam olarak bu kaostur.

Infrastructure as Code (Terraform) ise o şehrin 3 boyutlu CAD mimari planıdır.

Tek bir dosyada: - 1 adet PostgreSQL veritabanı - 2 adet yük dengeleyici - 1 adet S3 depolama alanı kod olarak yazılır. Şehir bir depremde yıkılsa bile tek bir komutla (`terraform apply`) 5 dakikada tüm şehri başka bir kıtada birebir yeniden inşa edebilirsiniz!

🧠 2. AI Çağında Yeni Zihniyet Modeli

Terraform Çalışma Döngüsü:

```
[main.tf Kodunu Yaz]
        |
        ▼
[1. terraform init] ──> Bulut sağlayıcı eklentilerini (AWS Provider) indirir.
        |
        ▼
[2. terraform plan] ──> "Neler değişecek?" (Önizleme yapar, ASLA değişiklik yapmaz!)
        |
        ▼
[3. terraform apply] ──> Gerçek buluta gider ve kaynakları inşa eder!
        |
        ▼
[terraform.tfstate] ──> Bulutun mevcut durumunu hafızasında tutan kilit dosya.
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: DevSecOps ve Terraform Mimarı
AWS altyapımız için Terraform (HCL) kodu yazacağız.

## GÖREV:
Aşağıdaki altyapıyı oluşturan modüler bir `main.tf` hazırla:
1. Şifrelenmiş (AES-256) ve public erişime kapalı bir AWS S3 bucket'ı.
2. Bu bucket'a dosya yüklendiğinde versiyonlamayı (versioning) aktif et.
3. Altyapıyı test etmek için `terraform plan` çıktısının nasıl okunacağını açıkla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Terraform: Güvensiz, şifreleme yok, public erişim açık!
resource "aws_s3_bucket" "insecure_bucket" {
  bucket = "sirket-gizli-veriler"
  # Şifreleme yok, versiyonlama yok!
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# Profesyonel ve Güvenli Terraform Mimarisi (main.tf)

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

# 1. S3 Depolama Alanı
resource "aws_s3_bucket" "secure_assets" {
  bucket        = "sirket-guvenli-depo-${var.environment}"
  force_destroy = false

  tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
}

# 2. Versiyonlama Koruması (Yanlışlıkla silinmeyi önler)
resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.secure_assets.id
  versioning_configuration {
    status = "Enabled"
  }
}

# 3. Sunucu Taraflı Şifreleme (AES-256)
resource "aws_s3_bucket_server_side_encryption_configuration" "crypto" {
  bucket = aws_s3_bucket.secure_assets.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

# 4. İnternete Tamamen Kapat (Block Public Access)
resource "aws_s3_bucket_public_access_block" "public_block" {
  bucket = aws_s3_bucket.secure_assets.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

💡 **Kod Analizi:** Altyapı kodla tanımlandığı için şifreleme, versiyonlama ve public erişim kilidi gibi güvenlik standartları zorunlu kılınmış; insan dalgınlığıyla açık bir S3 bucket bırakma riski sıfırlanmıştır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** `terraform.tfstate` dosyasını GitHub'a commit etmek (Şifreler ve API anahtarları içerebilir!).
  - **Çözüm:** State dosyasını daima AWS S3 ve DynamoDB kilitli uzak backend'de (Remote Backend) saklayın.
- **Tuzak:** `terraform plan` çıktısını okumadan körü körüne `terraform apply` çalıştırmak.
  - **Çözüm:** Plan çıktısındaki kırmızı silme (`- destroy`) uyarılarını mutlaka tek tek inceleyin.
- **Tuzak:** Konsoldan (ClickOps) elle değişiklik yapıp Terraform koduyla uyumsuzluk (Drift) yaratmak.
  - **Çözüm:** Bir kaynak Terraform ile yönetiliyorsa, konsoldan elle müdahaleyi kesinlikle yasaklayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 53: Infrastructure as Code - Terraform ile Bulutu Kodlamak*

- 🖥️ **Slayt 1:** ClickOps felaketi: Neden konsolda elle tıklamak yasaklanmalıdır?
- 🖥️ **Slayt 2:** IaC felsefesi: Altyapıyı Git ile versiyonlamak.
- 🖥️ **Slayt 3:** Terraform yaşam döngüsü: init, plan, apply, destroy.
- 🖥️ **Slayt 4:** Canlı Demo: 60 saniyede kodla AWS S3 bucket ayağa kaldırmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir `main.tf` dosyası oluşturun. `terraform init` ve `terraform plan` çalıştırın. Terminalde yeşil `+ create` satırlarını gösterin. `terraform apply` diyerek AWS'de (veya LocalStack'te) saniyeler içinde kaynağın oluştuğunu gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bilgisayarınıza Terraform kurun (`brew install terraform`) ve LocalStack veya AWS kullanarak basit bir depolama alanı oluşturan ilk `.tf` dosyanızı yazıp `terraform plan` alın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Terraform'da `terraform plan` komutunun temel işlevi nedir? - A) Buluttaki tüm sunucuları silmek - B) Yazılan kod ile bulutun mevcut durumunu karşılaştırarak, hangi kaynakların ekleneceğini, değiştirileceğini veya silineceğini buluta DOKUNMADAN güvenle önizlemek - C) Sadece fatura miktarını hesaplamak - D) Bilgisayarı kapatmak > **Doğru Yanıt:** B > **Açıklama:** `terraform plan`, altyapı üzerinde hiçbir yıkıcı işlem yapmadan önce geliştiriciye tam bir değişiklik önizlemesi (dry-run) sunar.

Soru 2: Terraform'un ürettiği `terraform.tfstate` dosyasını neden public bir Git reposuna commit etmemeliyiz? - A) Dosya çok büyük olduğu için - B) State dosyasının veritabanı şifreleri, API anahtarları gibi hassas bilgileri düz metin olarak içerebilmesi ve güvenlik açığı yaratması - C) Sadece Linux'ta çalışması - D) Kodun derlenmesini engellemesi > **Doğru Yanıt:** B > **Açıklama:** Terraform state dosyası buluttaki kaynakların tüm detaylarını (şifreler dahil) tutar; bu nedenle daima şifrelenmiş uzak backend'lerde (S3 vb.) saklanmalıdır.